

ObservableFlow v0.5: Trading Spatial Sensors for Temporal Readout in Reduced-Order Flow Reconstruction

Yaoharee Lahtee
Open Civil Science Initiative, Bangkok, Thailand

September 2026

[SimulatedData] Simulation=Yes

Abstract

Sensor placement is usually posed as a spatial selection problem: given a reduced representation of a physical system, choose a small set of measurements that preserves enough information for reconstruction or estimation. ObservableFlow v0.5 studies a different but related degree of freedom: a system may trade *spatial measurement channels* for *temporal readout depth*. Instead of asking only which sensors should be installed, the design variable is a pair (S, R) , where S is a selected set of measurement channels and R is the finite temporal depth used by the observer.

The frozen v0.5 implementation combines finite-horizon observability, rank and singular-value diagnostics, a stability-aware greedy search, an OpenFOAM adapter, reduced-order modelling, held-out reconstruction, and an external PySensors baseline. The final experiment uses one pinned OpenFOAM Foundation v13 cavity trajectory, 196 standardized scalar channels derived from a 7×7 pressure/velocity probe grid, and a common rank-8 POD basis learned on training data. Data are split chronologically into 60% training, 20% validation, and 20% untouched test. ObservableFlow selects three spatial channels at temporal depth four, while the validation-selected PySensors SSPOR/QR baseline uses eight static channels. Under the declared cost model, ObservableFlow has cost 4.0 versus 8.0 and lower final full-reference NRMSE in both the noiseless and 1%-noise tests. ObservableFlow therefore Pareto-dominates the pinned static baseline on the three declared final-test axes. However, its noiseless NRMSE is approximately 0.607, above the predeclared deployment gate of 0.50. The correct result is therefore benchmark-specific superiority on declared comparison axes, without deployment readiness or general market superiority.

1 Problem formulation

Let a reduced state evolve according to the affine discrete model

$$x_{t+1} = Ax_t + a, \tag{1}$$

and let candidate measurement channels satisfy

$$y_t = Cx_t + c. \tag{2}$$

A conventional sparse-sensor formulation chooses a subset of rows of C . ObservableFlow additionally permits the observer to retain a finite measurement history. For sensor set S and temporal depth R , the

finite-horizon observation matrix is

$$\mathcal{O}_{S,R} = \begin{bmatrix} C_S \\ C_S A \\ C_S A^2 \\ \vdots \\ C_S A^R \end{bmatrix}. \quad (3)$$

The basic structural requirement is

$$\text{rank}(\mathcal{O}_{S,R}) \geq r_{\text{target}}. \quad (4)$$

Rank is necessary but not sufficient. The preceding v0.4 benchmark exposed this directly: a one-channel temporal design reached the requested rank yet was catastrophically ill-conditioned and reconstructed poorly. v0.5 therefore continues searching after first rank saturation and introduces explicit conditioning and held-out reconstruction checks.

The structural score used during v0.5 search is

$$C_{\text{sensor}}(S) + C_{\text{depth}}(R) + 0.25 \log_{10} \kappa(\mathcal{O}_{S,R}), \quad (5)$$

with a declared condition-number ceiling. Full-rank candidates surviving structural search are evaluated on validation reconstruction. Final validation selection uses

$$C + 0.25 \log_{10} \kappa + 4E_{\text{val}} + 2E_{\text{val,noise}}. \quad (6)$$

These coefficients are benchmark choices, not universal physical constants.

2 Implementation architecture

ObservableFlow v0.5 is implemented as a Python package inside the Navier–Stokes readout repository. The relevant layers are:

1. **Core observability analysis.** Construction of finite-horizon measurement systems, numerical rank, singular values, condition number, and declared sensor/depth cost.
2. **Stability-aware search.** A deterministic greedy path is grown at every temporal depth. Before full rank, rank gain dominates the local choice; after full rank, the search continues so additional channels may improve conditioning. Every full-rank point is retained for application-level validation.
3. **OpenFOAM adapter.** Standard `postProcessing` outputs for scalar and vector probes are parsed, including restarted time segments.
4. **Reduced-order bridge.** Training snapshots are standardized using training statistics, projected to a POD basis, and fitted with affine reduced dynamics and measurement maps.
5. **Engineering validation.** Candidate designs are evaluated on held-out data in noiseless and noisy settings.
6. **External baseline.** The comparison uses the real `dynamicslab/pysensors` project pinned to a fixed commit, with SSPOR, native QR, and the exact same training POD basis supplied through the `PySensors Custom basis` interface.

The implementation deliberately separates structural selection from held-out validation. Test error is not part of the structural optimizer.

3 Frozen benchmark protocol

The final v0.5 evidence lane uses the following pinned setup:

- OpenFOAM Foundation v13 cavity tutorial at commit 18870c24d21c6b982e2cdec27b2f59738cca5f90;
- runtime image digest `sha256:a891bb2da102378efc956fb00ff05fc1c686edf38e8343fb87f3603e081ca524`;
- PySensors at commit 65400cd12e2f2b79e8a24d16852dd1371c14aa4e;
- 401 transient samples and 196 scalar channels from pressure and velocity;
- rank-8 POD carrying approximately 0.9998809 of the training energy under the standardized benchmark representation;
- chronological split of 240 training, 80 validation, and 81 test samples;
- Gaussian measurement noise with standard deviation 0.01 in training-standardized units;
- at most eight spatial channels and temporal depth at most twelve.

Training alone determines standardization, POD, reduced dynamics, measurement fits, and sensor rankings. Validation chooses the final ObservableFlow design and the final PySensors reconstruction method. The test segment is untouched until final evaluation.

The declared deployment gates are

$$\kappa \leq 1000, \quad E_{\text{clean}} \leq 0.50, \quad E_{\text{noise}} \leq 1.00. \quad (7)$$

No gate is relaxed after observing test performance.

4 Final test result

Table 1 reports the frozen untouched-test comparison.

Quantity	ObservableFlow v0.5	PySensors SSPOR/QR
Spatial channels	3	8
Temporal depth	4	0
Effective time samples/estimate	5	1
Reduced target rank	8	8
Achieved rank	8	8
Condition number	106.304	8.849
Declared total cost	4.0	8.0
Noiseless full-reference NRMSE	0.6073	0.7676
1%-noise full-reference NRMSE	0.6076	0.7676
Deployment pass	No	No

Table 1: Frozen v0.5 final-test comparison. “Full-reference” denotes the dense sampled 7×7 pressure/velocity surrogate used by the benchmark, not the native CFD mesh.

ObservableFlow selected two pressure channels and one y -velocity component: `p[p7]`, `p[p27]`, and `U[p42].y`. Under the declared cost model, the benchmark cost falls from 8.0 to 4.0, a 50% reduction.

Spatial channel count falls by 62.5%. Relative to the selected PySensors baseline, noiseless NRMSE falls by approximately 20.89% and noisy NRMSE by approximately 20.84%.

The frozen winner rule compares only

$$(C, E_{\text{clean}}, E_{\text{noise}}). \quad (8)$$

ObservableFlow Pareto-dominates if it is no worse in all three and strictly better in at least one. On this benchmark it is strictly better in all three, so it is the benchmark-specific Pareto winner.

This does not mean it is better on every engineering property. PySensors has substantially better conditioning in the selected comparison. ObservableFlow’s condition number remains below the declared ceiling but does not support a universal robustness claim.

5 Why the negative deployment result matters

The development sequence separates four claims that are often conflated:

$$\text{full rank} \not\Rightarrow \text{stable inversion} \not\Rightarrow \text{held-out reconstruction quality} \not\Rightarrow \text{deployment readiness}. \quad (9)$$

v0.4 exposed the first failure mode. v0.5 changed the search logic so the optimizer does not stop at first rank saturation, then used validation data to choose among full-rank candidates. The final design is much more stable than the v0.4 one-channel solution and performs better than the matched static baseline on the frozen comparison axes.

Nevertheless, the final clean NRMSE remains above 0.50. The benchmark is therefore frozen with this failure intact rather than tuned retrospectively until it crosses the gate. This preserves a clean distinction between method development and benchmark fitting.

6 Scope of the evidence

The frozen experiment establishes the following narrow points:

- a real OpenFOAM solver lane can feed ObservableFlow end to end;
- spatial sensor count and temporal depth can be optimized jointly in a reduced-order flow reconstruction workflow;
- stability-aware continuation beyond first full rank avoids the catastrophic rank-only design found in v0.4;
- under the declared cost model, matched representation, frozen split, and pinned PySensors SS-POR/QR baseline, ObservableFlow v0.5 Pareto-dominates on final cost, clean NRMSE, and noisy NRMSE;
- both methods fail the predeclared clean-error deployment gate.

The experiment does *not* establish general superiority to PySensors, commercial digital-twin products, or sparse-sensing methods more broadly. It does not establish industrial CAPEX/OPEX savings, robustness across Reynolds numbers or geometries, turbulent-flow performance, physical-hardware performance, closed-loop control performance, or reconstruction of the complete native CFD mesh. It also makes no claim about continuum Navier–Stokes existence or smoothness.

7 Reproducibility and closure

The frozen execution is GitHub Actions run 34487277000. The uploaded benchmark artifact is named `observableflow-v05-head2head` and has digest

`sha256:70eb6dceb36c55b2fb65ac6c1c8edc1e705a1c2a9476c627dab50c73483d9022`.

The committed final result is

`software/observableflow/benchmarks/openfoam13_cavity/results/head_to_head_v05.json`

with Git blob SHA `11cb7e7e01fe5436832c436a7311a268fb1623bb`. The canonical closure package also contains a machine-readable manifest and a separate adversarial closure audit.

ObservableFlow v0.5 is therefore closed as a **benchmark-specific research prototype**. The appropriate next evidentiary step is not continued tuning of the frozen cavity lane. A future v0.6 should preregister independent cases and regimes, retain external baselines, use physically motivated sensor/latency costs, and include genuinely unsteady or turbulent flow together with measured physical data when available.

Frozen concise claim. On one frozen OpenFOAM-13 cavity benchmark, ObservableFlow v0.5 used three spatial channels plus temporal depth four and Pareto-dominated a pinned PySensors SSPOR/QR static baseline on the benchmark’s declared acquisition-cost, noiseless-NRMSE, and 1%-noise-NRMSE axes of an untouched final test segment. Both methods failed the predeclared clean-NRMSE deployment gate. The result therefore supports the sensor–time design principle in this benchmark, not general market superiority or production readiness.